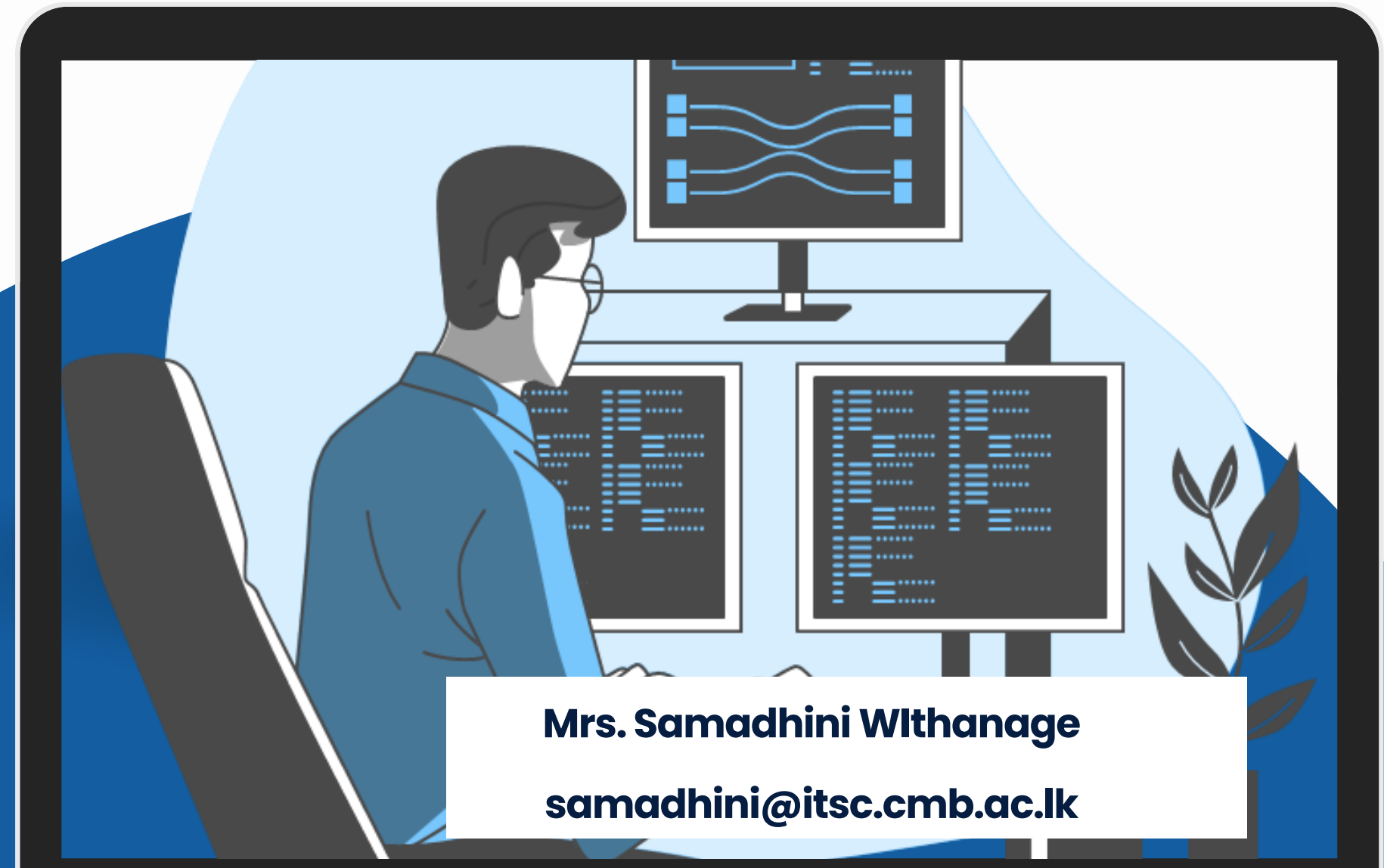


IT 3003

Advanced Programming Techniques



Mrs. Samadhini Wlthanage

samadhini@itsc.cmb.ac.lk

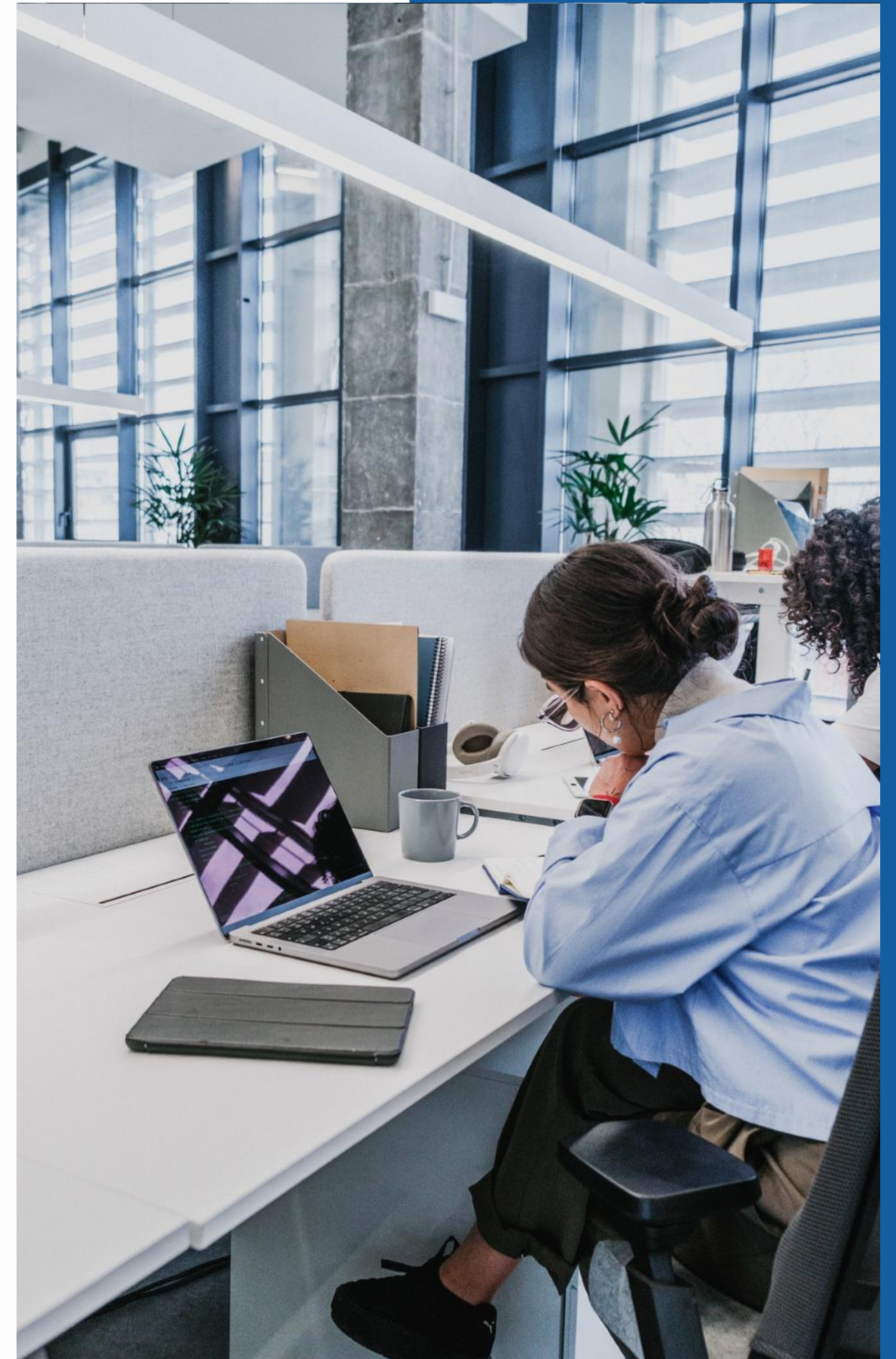
The background of the slide is a photograph of a modern office. In the foreground, a woman with curly hair is sitting at a desk, working on a laptop. To her right, a man in a light blue shirt is also working at a desk. In the background, another person is visible, and there are various office supplies like pens, paper, and a water bottle on the desks. The office has a clean, minimalist design with white walls and desks.

Lesson 1

Introduction to Programming

Lesson Outline

- ▶ What is a Programming Language
- ▶ Computer Programs
- ▶ Types of Programming Languages
- ▶ Compilation vs Interpretation
- ▶ Programming Paradigms
- ▶ Procedural vs Object-Oriented
- ▶ Debugging & Troubleshooting



What is a Programming Language?

- ▶ A programming language is a way to communicate with a computer.
- ▶ It allows humans to write instructions that computers can execute.
- ▶ Uses specific syntax (rules) and semantics (meaning)
- ▶ Artificial Language

Example : Java, Python, C++

Computer Programs

- ▶ A computer program is a set of instructions written using a programming language.
- ▶ Programs perform specific tasks.

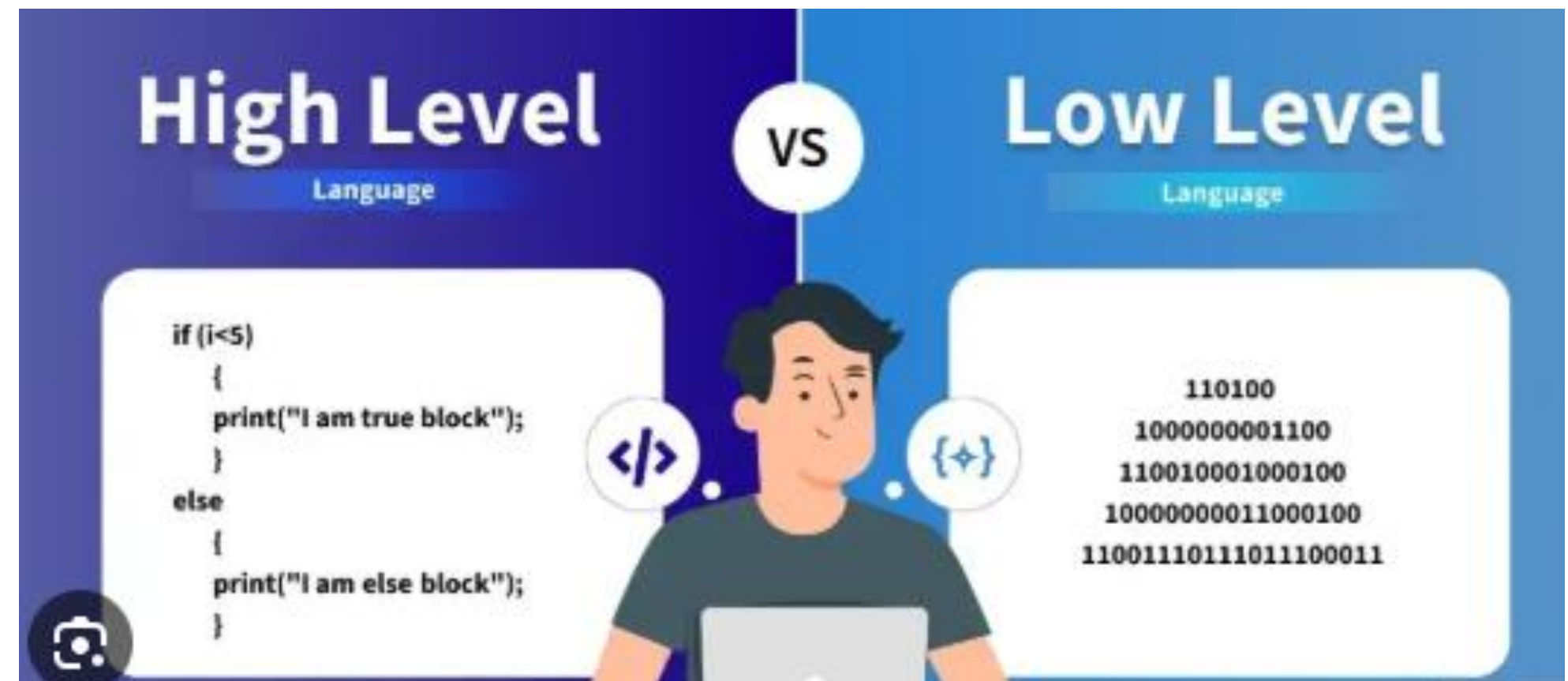
Example :

Calculator app,
Mobile apps
ATM systems



Types of Programming Languages

- 1 Low-Level Languages
- 2 High-Level Languages



Types of Programming Languages cont...

Low-Level Languages

- ▶ Low-level languages are **closer to the computer's hardware** and machine code.
- ▶ Fast but hard to understand.

Example :

Machine Language (Binary)
Only 0s and 1s

```
10110000 01100001
```

Assembly Language
Uses short codes (mnemonics)

```
MOV A, 61H
```

Types of Programming Languages cont...

Key Features of Low-Level Languages

- ▶ Very close to hardware (Directly interacts with CPU and memory.)
- ▶ Machine dependent (Different for each processor.)
- ▶ No need for heavy translation (Assembly uses an **assembler**.)

Advantages	Disadvantages
Very fast execution	Hard to learn and write
Efficient memory usage	Difficult to debug
Full control over hardware	Not portable

Types of Programming Languages cont...

High-Level Languages

- ▶ High-level languages are designed to be **easy for humans to read, write, and understand.**

Example :

Java
Python
C++
JavaScript

```
</> Python  
print("Hello World")
```

Types of Programming Languages cont...

Key Features of High-Level Languages

- ▶ Human-friendly syntax (They look similar to English)
- ▶ Easier to learn and use (You don't need to worry about hardware details)
- ▶ Portable (Platform Independent) (Programs can run on different systems with little or no changes.)
- ▶ Requires a translator (**Compiler** or **Interpreter** to convert into machine code)

Advantages	Disadvantages
Faster development	Slightly slower execution
Easier debugging	Less control over hardware
More readable code	
Suitable for large applications	

Types of Programming Languages cont...

Key Differences

Feature	High-Level Language	Low-Level Language
Readability	Easy (English-like)	Hard (binary/symbols)
Speed	Slower	Faster
Hardware Control	Limited	Full control
Portability	High	Low
Examples	Java, Python	Assembly, Machine Code

Types of Programming Languages cont...

When to Use Which

Apps
websites
software development
Most modern programming work

High-Level Languages

Operating systems
Embedded systems
Performance-critical tasks

Low-Level Languages

Compilation vs Interpretation



Compilation

Entire program translated at once

Produces an executable file

Faster execution

Example : Java



Interpretation

Code executed line-by-line

No separate executable file

Easier debugging

Example : Python

Programming Paradigms

Programming paradigms are a way to classify programming languages based on their features.

Same problem can be solved in different ways using different paradigms

- ▶ It allows humans to write instructions that computers can execute.
- ▶ Uses specific syntax (rules) and semantics (meaning)

Example : Java, Python, C++

Programming Paradigms Cont...

Main Paradigms

- ▶ Procedural
- ▶ Object-Oriented
- ▶ Functional
- ▶ Imperative

Programming Paradigms Cont...

Procedural Programming

▶ Programs are written as a **sequence of steps (procedures/functions)**.

▶ Key Features :

Step-by-step instructions

Uses functions/procedures

Top-down approach

Example : C

```
// C

void main() {
    int a = 5, b = 3;
    int sum = add(a, b);
}

int add(int x, int y) {
    return x + y;
}
```

Programming Paradigms Cont...

Object-Oriented Programming (OOP)

▶ Programs are built using **objects** (real-world entities).

▶ Key Features :

Class → blueprint

Object → real instance

Encapsulation → data hiding

Inheritance → reuse code

Polymorphism → many forms

Example : Java

```
</> Java

class Car {
    String color;

    void drive() {
        System.out.println("Car is moving");
    }
}
```


Debugging & Troubleshooting

Debugging

- ▶ Debugging is the process of finding and fixing errors (bugs) in a program.
(mainly for code errors)
- ▶ A *bug* is any mistake that causes :
 - Wrong output
 - Program crash
 - Unexpected behavior

Debugging & Troubleshooting Cont..

Troubleshooting

- ▶ Troubleshooting is a broader process of,
Identifying the cause of a problem and solving it
- ▶ Troubleshooting includes :
 - Code issues
 - System issues
 - Environment problems (software, libraries, etc.)

Debugging & Troubleshooting Cont..

Types of Errors



Syntax Errors (Compile-Time Errors)

Violating language rules

The program **won't compile** (won't run at all)

`</> Java`

```
public class Test {  
    public static void main(String[] args) {  
        System.out.println("Hello World") // missing semicolon  
    }  
}
```

Problem:

- Missing ; at the end
- Java compiler will show an error message

Debugging & Troubleshooting Cont..

Types of Errors Cont..



Runtime Errors

Occur while running

The program compiles successfully, but crashes during execution

`</>` Java

```
public class Test {  
    public static void main(String[] args) {  
        int x = 10;  
        int y = 0;  
        int result = x / y; // runtime error  
        System.out.println(result);  
    }  
}
```

Problem:

Division by zero → causes

ArithmeticException

Debugging & Troubleshooting Cont..

Types of Errors Cont..

▶ Logical Errors

Program runs but gives wrong output

Java

```
public class Test {  
    public static void main(String[] args) {  
        int a = 5;  
        int b = 3;  
        int sum = a * b; // wrong logic  
        System.out.println(sum);  
    }  
}
```

Problem:

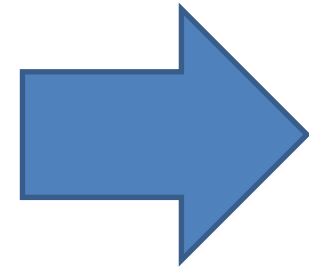
- Used * instead of +
- Output is wrong (15 instead of 8)

Debugging & Troubleshooting Cont..

Step-by-Step Debugging Approach

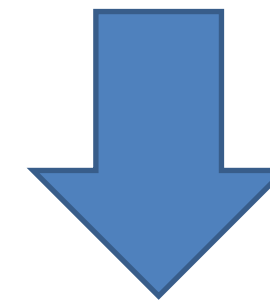
Step 1: Identify the Problem

- What is going wrong?
- Wrong output?
- Crash?



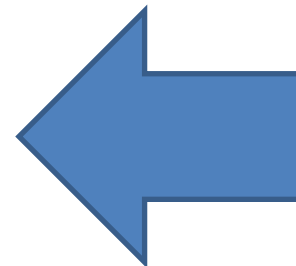
Step 2 : Reproduce the Error

- Run the program again
- Confirm the error happens consistently



Step 3: Read Error Messages

- Most languages give error details
- Don't ignore them—they often tell
 - Line number
 - Type of error



Step 4: Locate the Source

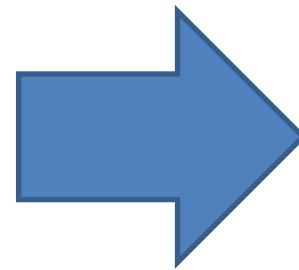
- Check the part of code causing the issue
- Focus on
 - Recent changes
 - Complex logic

Debugging & Troubleshooting Cont..

Step-by-Step Debugging Approach

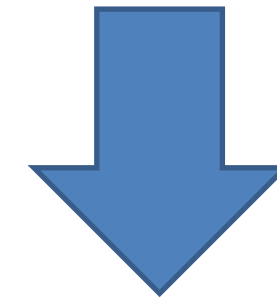
Step 5: Test Small Parts

- Break program into smaller sections
- Test each part separately
- This is called **isolation**



Step 6 : Fix the Issue

- Correct the mistake carefully
- Avoid changing too many things at once



Step 7: Test Again

- Run the program
- Check if:
 - Error is fixed
 - No new errors appear

Take-Home Exercise

Question 1

Explain the difference between **Procedural Programming** and **Object-Oriented Programming** using a real-world example.

*(Hint for students: Think about something like a **bank system** or **student system**.)*

Question 2

Consider the following problem: “Calculate the sum of numbers from 1 to 5.”

Write how this problem can be solved using the **Imperative approach**

Then explain how the same problem can be viewed in a **Functional approach**

(No need for full code—logic explanation is enough)

Summary

- ▶ What is a Programming Language
- ▶ Computer Programs
- ▶ Types of Programming Languages
- ▶ Compilation vs Interpretation
- ▶ Programming Paradigms
- ▶ Procedural vs Object-Oriented
- ▶ Debugging & Troubleshooting

The background is a blurred office scene with people working at desks, overlaid with a semi-transparent blue filter. A white rectangular box is centered on the image, containing the text "ANY QUESTIONS ?" in a bold, white, sans-serif font. The word "ANY" is on the top line, and "QUESTIONS ?" is on the bottom line.

**ANY
QUESTIONS ?**



THANK YOU!

